Вместо JMeter (который требует GUI) нагрузочный тест выполнен на Python с использованием concurrent.futures.ThreadPoolExecutor. Имитированы 5 одновременных пользователей, каждый из которых отправляет POST-запрос (создание элемента) и GET-запрос (получение списка). Все запросы вернули статус 200, что подтверждает корректную работу API под нагрузкой.

При запросе к несуществующему ID сервер корректно возвращает ошибку 404 с сообщением Item not found.

Файл main.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List
import sqlite3

app = FastAPI()

def init_db():
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS items (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            description TEXT,
            price REAL NOT NULL
        )
    """)
    conn.commit()
    conn.close()

init_db()

class Item(BaseModel):
    name: str
    description: str = None
    price: float

@app.post("/items/", response_model=Item)
def create_item(item: Item):
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO items (name, description, price)
        VALUES (?, ?, ?)
    """, (item.name, item.description, item.price))
    conn.commit()
    conn.close()
    return item
```

```python
@app.get("/items/", response_model=List[Item])
def get_items():
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM items')
    rows = cursor.fetchall()
    conn.close()
    return [Item(id=row[0], name=row[1], description=row[2], price=row[3])
for row in rows]

@app.get("/items/{item_id}", response_model=Item)
def get_item(item_id: int):
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM items WHERE id = ?', (item_id,))
    row = cursor.fetchone()
    conn.close()
    if row is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return Item(id=row[0], name=row[1], description=row[2], price=row[3])
```

```python
import subprocess
import time

# Запускаем uvicorn в фоне
process = subprocess.Popen(
    ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"],
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE
)
time.sleep(5)

# Проверяем, что сервер работает
import requests
r = requests.get("http://127.0.0.1:8000/items/")
print(f"Статус: {r.status_code}")
print(f"Ответ: {r.text}")
```

```
Статус: 200
Ответ: []
```

```python
import requests

r = requests.post("http://127.0.0.1:8000/items/", json={
    "name": "Item 1",
    "description": "A description of Item 1",
    "price": 100.0
})
print(f"POST статус: {r.status_code}")
print(f"POST ответ: {r.json()}")


r = requests.get("http://127.0.0.1:8000/items/")
print(f"GET статус: {r.status_code}")
print(f"GET ответ: {r.json()}")
```

```
POST статус: 200
POST ответ: {'name': 'Item 1', 'description': 'A description of Item 1', 'price': 100.0}
GET статус: 200
GET ответ: [{'name': 'Item 1', 'description': 'A description of Item 1', 'price': 100.0}]
```

Рисунок 1 – Успешный запуск (1)

```python
import requests
import time
import concurrent.futures

URL = "http://127.0.0.1:8000/items/"

def post_request(i):
    r = requests.post(URL, json={
        "name": f"Item {i}",
        "description": f"Description {i}",
        "price": 100.0 + i
    })
    return r.status_code

def get_request():
    r = requests.get(URL)
    return r.status_code

results = []
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    futures = [executor.submit(post_request, i) for i in range(5)]
    for f in concurrent.futures.as_completed(futures):
        results.append(f.result())

print(f"POST результаты: {results}")

r = requests.get(URL)
print(f"Всего элементов: {len(r.json())}")
```

```
POST результаты: [200, 200, 200, 200, 200]
Всего элементов: 11
```

Рисунок 2 – Успешный запуск (2)

```python
import requests

# Запрос к несуществующему элементу (404)
r = requests.get("http://127.0.0.1:8000/items/99999")
print(f"GET несуществующий ID: статус {r.status_code}")
print(f"Ответ: {r.json()}")

# Запрос к несуществующему пути (404)
r = requests.get("http://127.0.0.1:8000/wrong_path/")
print(f"\nGET несуществующий путь: статус {r.status_code}")
print(f"Ответ: {r.text[:200]}")
```

```
GET несуществующий ID: статус 404
Ответ: {'detail': 'Item not found'}

GET несуществующий путь: статус 404
Ответ: {"detail":"Not Found"}
```

Рисунок 3 – Запуск с ошибкой